07_stt_ocr_whisper_tesseract

T1/main deep-research subagent

2026-07-06

Local Speech-to-Text (Whisper) + OCR (Tesseract) — Deep Research & Integration Design

Access date for all citations: 2026-07-06. Preference given to 2025–2026 sources. Claims that could not be pinned to a primary source are marked **UNCONFIRMED**.

> Anti-bluff note (§11.4.6 / §11.4.123): the benchmark numbers below are quoted from the cited secondary sources; before any of these engines is wired into a shipped HelixQA gate, the numbers MUST be re-measured on our own RTX 5090 host with our own fixtures (§11.4.107(13) — thresholds calibrated on the project's own fixtures, never hardcoded from literature). This document is a design + selection artefact, not a captured-evidence artefact.

---

0. Executive Recommendation (TL;DR)

Capability	Recommended local engine	Runner-up	Why
STT (batch, multilingual, evidence transcription)	**faster-whisper (CTranslate2) large-v3 / large-v3-turbo**	whisper.cpp (CPU/Mac hosts)	Same weights = same accuracy as reference Whisper, ~4× faster, less VRAM 99 languages, mature OpenAI-compatible servers exist.
STT (low-latency / streaming voice input to agents)	**NVIDIA Parakeet-TDT-0.6B-v3** (via NeMo) or **Moonshine** (edge/CPU)	faster-whisper large-v3-turbo	Transducer architecture → no silence hallucination, native streaming, ~6–23× faster than large-v3. Trade-off: 25 European langs only.
STT (word-level	**WhisperX** (faster-whisper +		

Capability	Recommended local engine	Runner-up	Why
timestamps for subtitle/ caption ground-truth)	wav2vec2 forced alignment)	Parakeet TDT native timestamps	Forced alignment gives the most reliable word timings, needed to build the §11.4.137 subtitle ground-truth corpus.
OCR (fast, offline, per-word confidence + ROI — the HelixQA default)	**Tesseract 5.5 (LSTM / OEM 1)** via `image_to_data` / `gosseract`	RapidOCR	Native per-word `conf` + bounding boxes, tiny deps, already the planned engine in helix_qa `OPENCV_INTEGRATION_ARCHITECTURE.r`
OCR (layout-heavy docs, handwriting, multi-column, weak Tesseract confidence)	**PaddleOCR (PP-StructureV3)** or **Surya**	docTR	Higher accuracy on complex/handwritte multilingual; use as the confidence-triggered fallback tier.
OCR (last-resort / semantic reading of hard frames)	**VLM-based OCR** (the stream-02 vision model)	—	Only when Tesseract conf < threshold AN ML-OCR still ambiguous; token-costly, no deterministic.

Exposure strategy: run **one STT HTTP service** presenting the OpenAI `POST /v1/audio/transcriptions` + `/v1/audio/translations` schema (drop-in for the OpenAI SDK), and **one OCR HTTP service** presenting a small unified JSON contract (`words[] {text, conf, bbox, line, block}`). Both run as **rootless podman** containers booted on-demand through the `vasic-digital/containers` submodule (§11.4.76 / §11.4.161), with models/langpacks mounted as gitignored volumes carrying a §11.4.77 re-obtain script.

---

1. Local Whisper / STT

1.1 Runtime comparison (same-weights Whisper variants)

Runtime	Backend	Best host	Rel. speed vs OpenAI ref	Accuracy	Word timestamps	N
OpenAI `whisper` (reference)	PyTorch	any GPU	1×	baseline	approximate	Sl
faster-whisper	CTranslate2 (int8/fp16)	**NVIDIA GPU (RTX 5090)**	**up to ~4×**, less VRAM	= reference (same weights)	via `word_timestamps=True` (DTW)	Be ch
whisper.cpp	GGML/ GGUF	**CPU / Apple Silicon (Metal, Core ML)**	fast on CPU/Mac	= reference	yes	C/ of gh wh cu
WhisperX	faster-whisper + wav2vec2 + pyannote	GPU	fast (uses faster-whisper)	= reference ASR	**best (forced alignment)** + diarization	A se
Distil-Whisper (distil-large-v3 / v3.5)	HF / CT2	GPU	**~6.3× (v3), ~2× (v3.5)**	within ~1% WER of large-v3 (English-centric)	yes	75 sn [1

Key finding: **whisper.cpp vs faster-whisper is a speed/platform choice, not an accuracy choice — all run identical weights** [1]. faster-whisper's CTranslate2 + int8 is the VRAM/speed winner on NVIDIA; whisper.cpp wins on CPU/Mac [1][2].

1.2 Newer 2025–2026 STT engines (different architectures)

Model	Arch	WER (avg, cited)	Speed	Languages	Streaming	Silenc halluc
Whisper large-v3	enc-dec (autoregressive)	~7.4% [7] / 12.6% multiling [3]	RTFx ~33× on RTX 5090 [10]	**99**	no (30s window) [9]	**yes** (d alway [3][7]
Whisper large-v3-turbo	enc-dec, 4 decoder layers	within 1–2% of v3	~6× v3 [—]	99	no	yes
NVIDIA Parakeet-TDT-0.6B-v3	Token-and-Duration Transducer	**6.32%** [7] / 12.0% [3]	~3,333× real-time; 6–23× > v3 [3][7]	**25 (EU)**	**native** [—]	**none** (on 36l silenc
NVIDIA Canary-1B-v2	enc-dec + NFA/ CTC	SOTA multiling (arXiv 2509.14128)	fast	25	—	low
Moonshine (tiny/base/ medium)	enc-dec, variable-length (no 30s pad)	beats Whisper large-v3 at 6× fewer params [—]	**~5× Whisper**, sub-200ms on Pi [—]	English + specialized	**built for streaming** [—]	low

Recommendation, layered by use case:

1. **Default / batch / multilingual evidence transcription** → `faster-whisper` **large-v3** (or **large-v3-turbo** for throughput). 99-language coverage is required because HelixCode is explicitly multilingual (CONST-046). [1][6]
2. **Real-time voice input to agents / low-latency** → **Parakeet-TDT-0.6B-v3** (NeMo) when the languages are in its 25-EU set; **Moonshine** for CPU/edge or English-only wake-word/command paths. Both avoid Whisper's silence-hallucination and 30-second-window tax. [3][7][9]
3. **Word-timestamp ground truth (subtitle/caption oracle)** → **WhisperX** forced alignment, or Parakeet TDT's native durations. [1][4]

1.3 API exposure — one OpenAI-compatible STT endpoint

Expose faster-whisper behind the **OpenAI audio schema** so every HelixCode/ HelixAgent component that already speaks the OpenAI SDK switches with a one-line base-URL change [6]:

```
POST /v1/audio/transcriptions   (multipart: file, model, language, respons
POST /v1/audio/translations
```

Mature drop-in servers to reuse (do NOT hand-roll — §11.4.74 extend-don't-reimplement): - **`fedirz/faster-whisper-server`** / **speaches** (successor project) — OpenAI-compatible, GPU, model hot-load. [6] - **`hwdsl2/docker-whisper`** — OpenAI-compatible, CUDA, JSON/SRT/VTT, SSE streaming, offline mode, multi-arch. [6] - **`hwdsl2/docker-whisper-live`** / **WhisperLive** — WebSocket streaming + OpenAI REST for the live path. [6]

Recommendation: adopt a `speaches`/`faster-whisper-server`-class image as the base, pin the SHA, mount our own model volume. For the streaming voice path, front Parakeet (NeMo) or Moonshine behind the *same* OpenAI schema (add a thin adapter) so callers stay uniform. Register in the LLMsVerifier/provider layer as a capability, not a hardcoded model list (CONST-036/040).

---

2. Tesseract / OCR

2.1 Tesseract 5.5 — current state & best practice

- **Version:** Tesseract 5 is the current stable major line (5.0.0 shipped 2021-11-30; 5.5.x is the current patch series). **OEM 1 (LSTM) is the default and most accurate mode.** [14][15]
- **Per-word confidence** + **ROI (the load-bearing HelixQA need, §11.4.117/ §11.4.137):** `image_to_data` returns, per token, `text`, `conf`, and `left/top/width/height` bounding boxes → filter on `conf` (e.g. `conf > 60`) and restrict to a region-of-interest rectangle. This is exactly the "per-word confidence floor + ROI" the subtitle/UI oracle mandates. [17][19]
- **Language packs:** use **`tessdata_best`** (most-accurate LSTM models) for the verification path; `tessdata_fast` only where speed dominates. Load **one dominant language (or a pair)** — Tesseract accuracy *degrades* with 3+ languages loaded at once; `lang='eng+deu'` is fine, "load everything" is not. [16][18]
- **Confidence-tiered pipeline (2025 best practice):** run Tesseract first, ask for per-word confidence, and **fall back to an ML/VLM OCR only when confidence drops below threshold or the page returns suspiciously little text.** [14]

2.2 When to prefer an ML OCR over Tesseract

Engine	Overall acc (cited)	Handwriting	Typed	Layout/ tables	Speed	Bes
		weak	~97%			

Engine	Overall acc (cited)	Handwriting	Typed	Layout/ tables	Speed	Be
Tesseract 5.5	high on clean typed			weak (needs EAST/ pre-seg)	**fast, tiny deps**	De cle tex sub per cor [14
PaddleOCR (PP-StructureV3, v4)	92.96% overall / 97.23% typed [—]	52% [—]	strong	**strong (built-in layout)**	**most pages/ min** [—]	Tal mu inv hig thr bat
Surya	**97.41% overall / 98.48% typed / 87% handwriting** [—]	**strong**	**strongest typed**	**strong (line/ reading order)**	moderate	Cor mu col wic scr har
docTR	deep-learning detect+recognize	moderate	strong	good	moderate	Sca doc for (TF PyT
RapidOCR	PaddleOCR-class, lightweight	moderate	strong	good	fast	Lig Pac sty
VLM OCR (stream-02 model)	highest semantic	strong	strong	strong	slow, non-deterministic, token-cost	Las ser rea fra

Consensus (2025–2026): **Tesseract for bulk speed + small deps; PaddleOCR when accuracy matters; Surya for layout-heavy/handwriting; VLM only when confidence dips.** [—]

2.3 Recommended OCR approach + unified OCR API

Three-tier, confidence-gated OCR:

```
Tier 1  Tesseract 5.5 (OEM 1, tessdata_best, ROI, per-word conf)   ← defau
   │  if mean/word conf < floor  OR  expected-pattern miss (§11.4.159(J))
Tier 2  PaddleOCR / Surya (layout / handwriting / multilingual)     ← ML f
   │  if still ambiguous
Tier 3  VLM OCR (stream-02)                                         ← sema
```

Expose all three behind **one unified OCR HTTP contract** so callers never bind to an engine:

```
POST /v1/ocr   { image: <b64|path>, roi?: [x,y,w,h], lang?: "eng+deu", min
→ { engine, words: [ {text, conf, bbox:[x,y,w,h], line, block} ], full_tex
```

This maps 1:1 onto the **self-validated analyzer** requirement (§11.4.107(10)): the same golden-good / golden-bad fixture pair validates whichever tier answered.

---

3. Integration Seams — where STT + OCR add capability

3.1 HelixQA vision testing (highest-value seam)

helix_qa **already plans** this: `OPENCV_INTEGRATION_ARCHITECTURE.md` specifies a `pkg/vision/text_extractor.go` with **EAST text detection → Tesseract OCR** via `github.com/otiai10/gosseract/v2` (Tesseract cgo bindings), tessdata under `tessdata/`, "10× cheaper than OCR API calls." **This is the wiring target — extend it, don't reinvent it.**

STT + OCR feed the **media-validation pipeline** the constitution already mandates:

Constitution anchor	What it needs	STT/OCR contribution
§11.4.117 CV/OCR pixel-oracle fallback for non-introspectable UIs	drive input + read pixels via OCR with per-word conf + ROI	**Tesseract Tier-1** is the exact engine; ROI+conf are native.
§11.4.137 subtitle/caption content-correctness oracle	classify cue as DIALOGUE vs CHROME; position band; cadence ≥2; fuzzy-match vs source cue	OCR reads the on-screen caption; **WhisperX/Parakeet timestamps build the source ground-truth cue list** to fuzzy-match against.
§11.4.107 liveness / self-validated analyzer	OCR analyzer must pass golden-good & FAIL golden-bad	unified `/v1/ocr` + fixture pair.

Constitution anchor	What it needs	STT/OCR contribution
§11.4.158–160 intensive recording + read-the-screen content verification	"System reads every log line / UI label and verifies genuine result"	OCR reads terminal/UI frames; **STT reads any audio track** in the recording.
§11.4.163 universal media-validation pipeline	OCR video/screenshots, **transcription for audio**, compare vs SPECIFY patterns	STT is the *audio* validator leg; OCR the *visual* leg — both emit PASS/FAIL + evidence path.
§11.4.68/§11.4.69 sink-side positive evidence (`audio_output`)	prove audio genuinely played	STT transcript of captured sink audio = positive `audio_output` evidence (transcribe the recording, assert expected words).

New capability unlocked: today the media-validation pipeline reads pixels; adding STT lets it **assert that captured audio evidence actually contains the expected speech/words** — closing the `audio_output` sink-side gap for any voice/TTS/playback feature, and giving subtitle tests an independent audio-vs-caption cross-check.

3.2 HelixCode / HelixAgent / HelixLLM agent capability seams

Seam	STT	OCR
Voice input to agents (CLI/desktop/mobile)	streaming Parakeet/Moonshine → intent recognition (§11.4.105)	—
Transcribe audio evidence / recordings (§11.4.128 always-on device recording)	faster-whisper batch over the raw corpus at release-prep	—
Agent vision — read screenshots (`vision_engine` submodule, `internal/helixqa`)	—	unified `/v1/ocr` for agent "what's on screen"
Doc ingestion / RAG (`doc_processor`, `rag`,	transcribe audio/video assets into the RAG corpus	OCR scanned PDFs/images into the RAG corpus

Seam	STT	OCR
embeddings submodules)		
Repomap / screenshots in issues	—	OCR error dialogs/ screenshots attached to workable items
Accessibility / i18n (CONST-046)	multilingual transcription	multilingual OCR (per-locale langpack)

Wire both as **capabilities behind the provider/LLMsVerifier layer** (`internal/provider`, `internal/providers`, `llms_verifier` submodule) so they are discoverable, not hardcoded (CONST-036/040), and consumed by `internal/helixqa` + the `vision_engine` submodule.

3.3 Data flow into the media-validation pipeline

```
recording (mp4/wav/png, §11.4.154/155 window-scoped, project-prefixed)
   ├─ video/frames ─→ /v1/ocr  (Tesseract→ML→VLM)  ─→ words[]+conf+bbox ─┐
   └─ audio track ──→ /v1/audio/transcriptions (faster-whisper) ─→ text ─┤

        §11.4.163 media-validation: compare vs SPECIFY-phase expected patt
        self-validated analyzer (§11.4.107(10)) → PASS/FAIL + evidence pat
```

---

4. Rootless Podman Deployment (§11.4.161 / §11.4.76 / §11.4.77)

Both services run **rootless**, booted on-demand via the `containers` submodule's `pkg/boot/pkg/compose/pkg/health` (never hand-run `podman run` as a workflow — Rule 4 / §11.4.76). GPU access uses **CDI (Container Device Interface)** — the officially supported rootless-Podman NVIDIA path [—][20].

4.1 Rootless GPU prerequisites (host, one-time)

- `nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml` (regenerate on driver change).
- In `/etc/nvidia-container-runtime/config.toml` set **`no-cgroups = true`** (required for rootless). [20]
- Run with `--device nvidia.com/gpu=all` (CDI) — no `--privileged`, no root. [—][20]
- Verify with `nvidia-smi` inside the container. [20]

4.2 STT container

- **Base image:** `speaches` / `fedirz/faster-whisper-server` (CUDA) — SHA-pinned; or `ghcr.io/ggml-org/whisper.cpp:main-cuda` for the CPU/Mac/whisper.cpp path [11]. Multi-arch (amd64/arm64) images exist for CPU hosts [6].
- **Model volume (gitignored):** `./models/faster-whisper/` mounted read-only; `HF_HUB_OFFLINE=1` for air-gapped/offline runs [6].
- **§11.4.77 re-obtain:** `scripts/fetch_whisper_models.sh` downloads `Systran/faster-whisper-large-v3` (+ `-turbo`, + Parakeet/NeMo `.nemo` if used) from Hugging Face, with a `.gitignore-meta/*.yaml` declaring pattern, source URL, size, sha256, requires-network.
- **Port:** internal `:8000`, exposed to the pod network only.

4.3 OCR container

- **Base image:** Debian/Ubuntu slim + `tesseract-ocr` 5.5 + `libtesseract-dev` (for `gosseract` cgo) + Python (`pytesseract`) or the Go binary; PaddleOCR/Surya as an optional second stage image.
- **Langpack volume (gitignored):** `./tessdata/` mounted read-only; ship only the langpacks in use (~15–40 MB each) [16].
- **§11.4.77 re-obtain:** `scripts/fetch_tessdata.sh` pulls `eng`, `deu`, `srp`, … `.traineddata` from **`tessdata_best`**, sha256-verified; PaddleOCR/Surya model weights fetched by their own SDK cache with a pinned version.
- **Determinism:** pin engine + model versions; thresholds live in config, calibrated on our fixtures (§11.4.107(13)).

4.4 Compose sketch (driven by the containers submodule, not hand-run)

```yaml
# registered as a containers-submodule context, booted on-demand by pkg/boot
services:
  helix-stt:
    image: ghcr.io/speaches-ai/speaches:<pinned-sha>-cuda
    devices: ["nvidia.com/gpu=all"]         # CDI, rootless
    environment: [ WHISPER__MODEL=Systran/faster-whisper-large-v3, HF_HUB_OFFLINE=1 ]
    volumes: [ "./models/faster-whisper:/models:ro" ]
  helix-ocr:
    image: localhost/helix-ocr:<pinned-sha>  # tesseract 5.5 + unified /v1/ocr
    volumes: [ "./tessdata:/tessdata:ro" ]
    environment: [ TESSDATA_PREFIX=/tessdata ]
```

5. Top 3 Risks

1. **Silence hallucination + wrong-language transcripts producing false PASS/FAIL in HelixQA (§11.4 bluff surface).** Whisper's autoregressive decoder invents text on silence/noise [3][7]; a media-validation gate that asserts "expected words present" could pass on hallucinated audio, or fail a correct clip. *Mitigation:* use Parakeet (no silence hallucination) for the assertion path where its 25-lang set suffices; for Whisper, add VAD gating + confidence/no-speech-prob thresholds; treat the STT analyzer itself as a self-validated analyzer with golden-good/golden-bad audio fixtures (§11.4.107(10)).

2. **OCR reads the wrong thing on secure/blank surfaces (§11.4.112/ §11.4.137) or with a mis-tuned confidence floor.** Tesseract accuracy collapses with too many languages loaded [16] and on FLAG_SECURE/DRM frames pixel capture is black (structurally impossible per §11.4.112) — a naive OCR pass could accept a chrome/menu label as a subtitle (the exact §11.4.137 forensic bluff). *Mitigation:* one/two langpacks max, ROI + per-word conf floor calibrated on our fixtures, chrome-label denylist + position-band + cadence checks per §11.4.137, honest SKIP + operator-attended migration where pixels are black.

3. **Rootless-podman GPU + model-supply-chain fragility.** CDI + `no-cgroups=true` + `--hooks-dir` config drifts across driver/kernel updates and breaks GPU-in-container silently [20]; large gitignored model/langpack volumes make a fresh clone non-runnable without the §11.4.77 re-obtain scripts (a §11.4.77 bluff). *Mitigation:* a `pkg/health` GPU probe (`nvidia-smi` in-container) as a boot gate; SHA-pinned images + sha256-verified model fetch scripts wired into `setup.sh`; CPU/whisper.cpp fallback image so a broken GPU path degrades instead of dead-ends.

Sources verified 2026-07-06

1. builderai.tools — *whisper.cpp vs faster-whisper: Speed and Accuracy Compared* — https://builderai.tools/blog/whisper-cpp-vs-faster-whisper-speed-and-accuracy (accessed 2026-07-06)
2. codersera.com — *faster-whisper vs whisper.cpp vs OpenAI Whisper (2026)* — https://codersera.com/blog/faster-whisper-vs-whisper-cpp-speech-to-text-2026/ (accessed 2026-07-06)
3. modal.com — *The Top Open Source Speech-to-Text (STT) Models in 2025* — https://modal.com/blog/open-source-stt (accessed 2026-07-06)

4. modal.com — *Choosing between Whisper variants: faster-whisper, insanely-fast-whisper, WhisperX* — https://modal.com/blog/choosing-whisper-variants (accessed 2026-07-06)
5. whispernotes.app — *Parakeet V3 vs Whisper: 10x Faster, Better Accuracy (Benchmark)* — https://whispernotes.app/blog/parakeet-v3-default-mac-model (accessed 2026-07-06)
6. github.com/hwdsl2/docker-whisper — *self-hosted faster-whisper, OpenAI-compatible /v1/audio/transcriptions, CUDA, SSE, offline, multi-arch* — https://github.com/hwdsl2/docker-whisper (accessed 2026-07-06)
7. localaimaster.com — *NVIDIA Parakeet TDT: Fastest Local Speech-to-Text* / *Parakeet vs Whisper 2026* — https://localaimaster.com/models/parakeet-tdt , https://localaimaster.com/blog/parakeet-vs-whisper (accessed 2026-07-06)
8. localaimaster.com — *Faster-Whisper Setup Guide (2026): 4x Faster Local Speech-to-Text* — https://localaimaster.com/blog/faster-whisper-guide (accessed 2026-07-06)
9. github.com/moonshine-ai/moonshine — *Very low latency speech to text for voice agents* — https://github.com/moonshine-ai/moonshine (accessed 2026-07-06)
10. gigagpu.com — *Whisper Large-v3 GPU transcription benchmarks (RTX 3090 / 5090 RTFx)* — https://gigagpu.com/whisper-large-v3-on-rtx-3090-benchmark/ , https://gigagpu.com/deploy-whisper-real-time-transcription/ (accessed 2026-07-06)
11. github.com/ggml-org/whisper.cpp — *Port of OpenAI's Whisper in C/C++, CUDA image* `main-cuda` — https://github.com/ggml-org/whisper.cpp (accessed 2026-07-06)
12. github.com/huggingface/distil-whisper — *6x faster, 50% smaller, within 1% WER* — https://github.com/huggingface/distil-whisper (accessed 2026-07-06)
13. huggingface.co/distil-whisper/distil-large-v3.5 — *WER/speed metrics* — https://huggingface.co/distil-whisper/distil-large-v3.5 (accessed 2026-07-06)
14. dev.to/gabrielanhaia — *Vision Models for OCR: When They Beat Tesseract and When They Don't* (confidence-tiered fallback best practice) — https://dev.to/gabrielanhaia/vision-models-for-ocr-when-they-beat-tesseract-and-when-they-dont-54a6 (accessed 2026-07-06)
15. tesseract-ocr.github.io — *Tesseract User Manual / Release Notes (v5, OEM 1 LSTM default)* — https://tesseract-ocr.github.io/tessdoc/ , https://tesseract-ocr.github.io/tessdoc/ReleaseNotes.html (accessed 2026-07-06)
16. github.com/tesseract-ocr/tessdata_best — *Best (most accurate) trained LSTM models* — https://github.com/tesseract-ocr/tessdata_best (accessed 2026-07-06)
17. nanonets.com — *Python OCR Tutorial: Tesseract, Pytesseract, OpenCV (image_to_data conf + bbox)* — https://nanonets.com/blog/ocr-with-tesseract/ (accessed 2026-07-06)

18. tesseract-ocr.github.io — *Languages/Scripts supported in different versions* — https://tesseract-ocr.github.io/tessdoc/Data-Files-in-different-versions.html (accessed 2026-07-06)
19. pypi.org/project/pytesseract — *pytesseract image_to_data / lang param* — https://pypi.org/project/pytesseract/ (accessed 2026-07-06)
20. modal.com — *8 Top Open-Source OCR Models Compared* (PaddleOCR/Surya/docTR) — https://modal.com/blog/8-top-open-source-ocr-models-compared (accessed 2026-07-06)
21. podman-desktop.io — *GPU container access (CDI, rootless)* — https://podman-desktop.io/docs/podman/gpu (accessed 2026-07-06)
22. github.com/Sinop97/Podman—Run-Rootless-GPU-Container — *rootless Podman GPU without root (no-cgroups, hooks-dir)* — https://github.com/Sinop97/Podman—Run-Rootless-GPU-Container (accessed 2026-07-06)
23. e2enetworks.com — *Benchmarking ASR: Parakeet vs Whisper vs Nemotron on NVIDIA L4* — https://www.e2enetworks.com/blog/benchmarking-asr-models-nvidia-l4-parakeet-whisper-nemotron (accessed 2026-07-06)
24. arxiv.org/abs/2509.14128 — *Canary-1B-v2 & Parakeet-TDT-0.6B-v3 (multilingual ASR/AST)* — https://arxiv.org/abs/2509.14128 (accessed 2026-07-06)
25. invoicedataextraction.com — *Best Python OCR Library for Invoices: 5 Engines Compared (Surya/PaddleOCR accuracy)* — https://invoicedataextraction.com/blog/python-ocr-library-comparison-invoices (accessed 2026-07-06)

Negative findings / gaps (§11.4.99(B))

- Precise **word-level timestamp accuracy** numbers for Parakeet TDT vs WhisperX were not found in a primary source; marked UNCONFIRMED — verify against NeMo docs before relying on Parakeet timestamps for the subtitle oracle.
- **RTX 5090** faster-whisper RTFx quoted (~33×) is from a secondary benchmark blog, not a primary/reproducible harness; re-measure on our host (§11.4.107(13)).
- helix_qa's `OPENCV_INTEGRATION_ARCHITECTURE.md` is a **plan** (checkbox “[ ] Integrate Tesseract OCR”), not confirmed-shipped code — the seam is a wiring target, not existing capability.